

# EE5516 - Mini project report

## Fixed-point Inverse Square root operation using Newton-Raphson

D Priyesh Narayana

Roll no: 152502007

*M.Tech in System-on-Chip Design*

*Indian Institute of Technology, Palakkad*

152502007@smail.iitpkd.ac.in

Kaushik Balaji

Roll no: 152502010

*M.Tech in System-on-Chip Design*

*Indian Institute of Technology, Palakkad*

152502010@smail.iitpkd.ac.in

### Abstract

The Newton-Raphson (NR) method is widely used for computing the inverse square root because of its fast convergence and hardware efficiency [1]. Unlike traditional approaches such as CORDIC-based reciprocal and square root computation, which suffer from high latency and larger resource usage, NR is an iterative method where each iteration approximately doubles the precision, significantly reducing computation time.

Additionally, the method is well suited for both hardware and software implementations due to its reliance on simple arithmetic operations. This makes it particularly effective in real-time systems where both speed and resource efficiency are critical. Historically, optimized variants of the NR-based inverse square root have been used in performance-critical applications such as real-time 3D graphics rendering, demonstrating its practical importance beyond theoretical efficiency.

### I. INTRODUCTION

The NR algorithm replaces the usage of expensive memory based lookup tables, by just using multipliers and simple arithmetic units. The speed advantage of NR mainly comes from:

- Its **quadratic convergence**, means fewer iterations are required for high accuracy
- the ability to start from **good approximation**, so minimizing iterations further
- **Hardware pipelining** is possible, so can have high throughput architecture

Another important advantage of NR-based implementations is the reduction or elimination of pre- and post-processing steps such as input scaling and coefficient storage, making the design more flexible and easier to integrate as a reusable hardware module.

The method is extensively used in digital signal processing (DSP) systems, especially in operations like **matrix inversion**, which appear in applications such as telecommunications and EEG signal processing. In 3D graphics, inverse square root is used constantly for **vector normalization**, which is needed for lighting calculations, physics and camera transformations.

A well-known practical example is its use in early real-time graphics engines such as Quake III Arena [2], where a fast inverse square root implementation combined a clever initial approximation with a single NR iteration. This approach provided sufficient numerical accuracy while significantly reducing computational cost, enabling high frame rates on limited hardware.

### II. ARCHITECTURE DESIGN

The proposed system implements a pipelined inverse square root computation using NR method. The architecture is composed of an initial approximation stage followed by multiple NR iteration stages.

The system has the following input and output signals:

- **a\_in** – Input fixed-point number whose inverse square root is to be computed
- **clk** – System clock
- **rst** – Synchronous reset signal

- **result** – Fixed-point representation of  $\frac{1}{\sqrt{a}}$

The overall architecture consists of the following main modules:

- **initial\_approx** – Generates an initial estimate  $x_0$  using Leading One Detection (LOD)
- **LOD** – Detects the position of the most significant '1' in the input
- **NR\_stage\_pipeline** – Performs one Newton-Raphson iteration in a pipelined manner
- **top\_pipelined** – Chains multiple NR stages to improve precision
- **top\_InvSqrt** – Top-level module integrating approximation and iteration stages

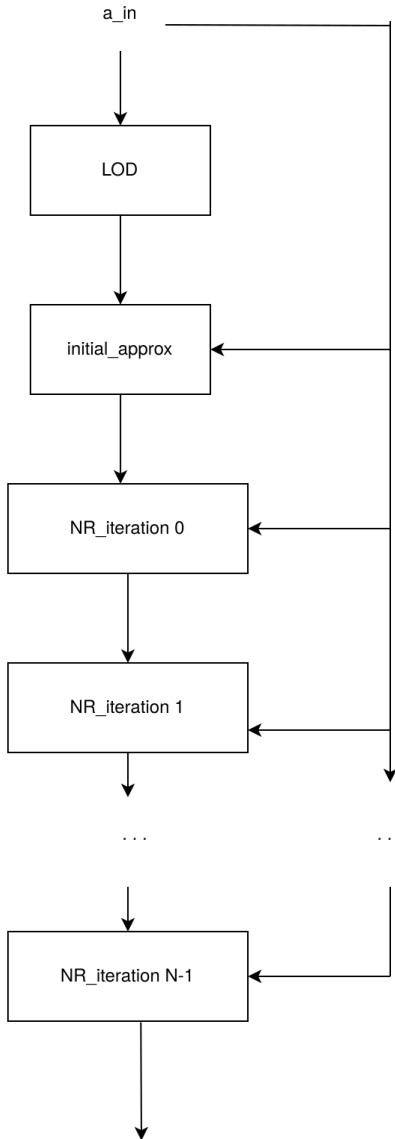


Fig. 1. Block diagram for the design

#### A. Initial Approximation Stage

The **initial\_approx** module computes the initial guess  $x_0$  based on the leading one position of the input  $a$ . The **LOD** module produces:

- **lod\_out** – One-hot representation of the leading '1'
- **lod\_index** – Index of the leading '1'

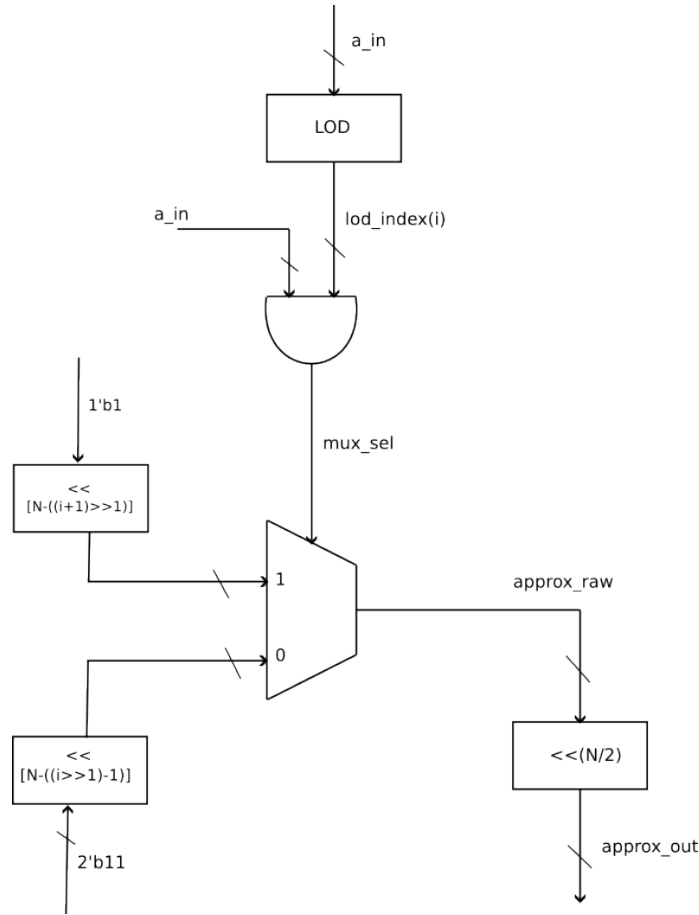


Fig. 2. initial\_approx

Using this, the approximation is generated as:

$$x_0 = \begin{cases} 2^{-\frac{j+1}{2}} & \text{if } j \text{ is odd} \\ 2^{-\left(\frac{j}{2}+1\right)} + 2^{-\left(\frac{j}{2}+2\right)} & \text{if } j \text{ is even} \end{cases} \quad (1)$$

where  $j$  is the index of the leading one. The result is stored in **approx\_out** after appropriate scaling.

#### B. Leading One Detector (LOD)

The **LOD** module determines the most significant set bit using:

- 1) Bit reversal of input
- 2) Two's complement operation
- 3) Bitwise AND to isolate the leading one

This isolates the least significant 1 in the reversed domain, which corresponds to the most significant 1 in the original input.

Intermediate signals include:

- **BR\_x** – Bit-reversed input
- **NOT\_out** – Bitwise inversion
- **adder\_out** – Two's complement result
- **AND\_out** – Isolated leading one (in reversed domain)

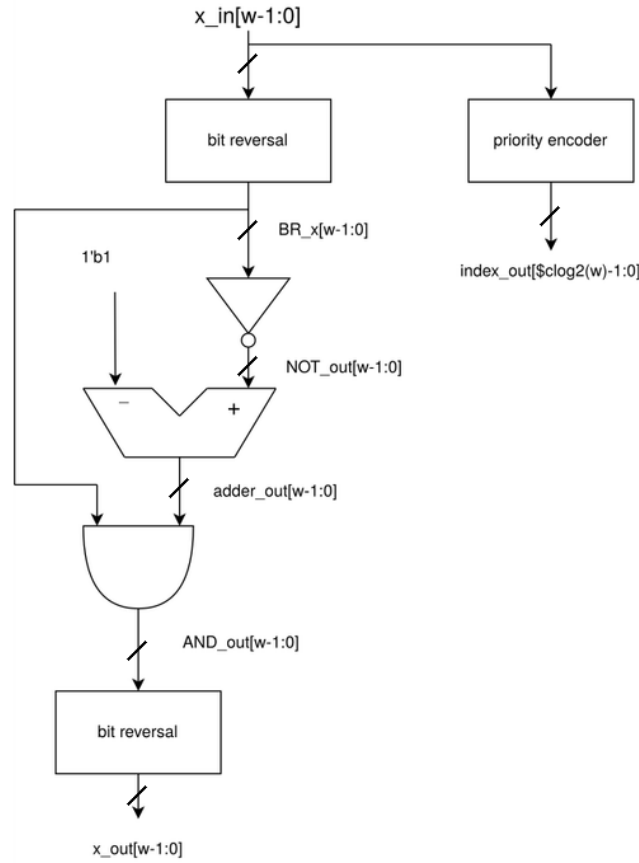


Fig. 3. LOD

### C. Newton-Raphson Iteration Stage

Each **NR\_stage\_pipeline** module computes one iteration of:

$$x_{i+1} = \frac{x_i}{2} (3 - ax_i^2) \quad (2)$$

In this design, Rounding was applied in selected stages ( $x^2$  and final multiplication) to improve accuracy, while truncation was used in intermediate steps to avoid bias and reduce hardware cost. Truncating the output in every stage will reduce the complexity of the design a bit, but it causes accumulation of error and the value doesn't converge properly within the given iterations.

The computation is divided into three pipeline stages:

- **Stage 1:** Compute  $x_i^2$
- **Stage 2:** Compute  $(3 - ax_i^2)$
- **Stage 3:** Compute final update  $x_{i+1}$

The NR iteration was split into 3 pipeline stages to balance critical path delay across multipliers and improve throughput.

Intermediate signals include:

- **x\_sq\_full, x\_sq\_scaled** – Squared value of input
- **ax2\_full** – Product  $a \cdot x_i^2$
- **term\_full** – Intermediate subtraction result
- **mult\_full, mult\_scaled** – Final multiplication and scaling

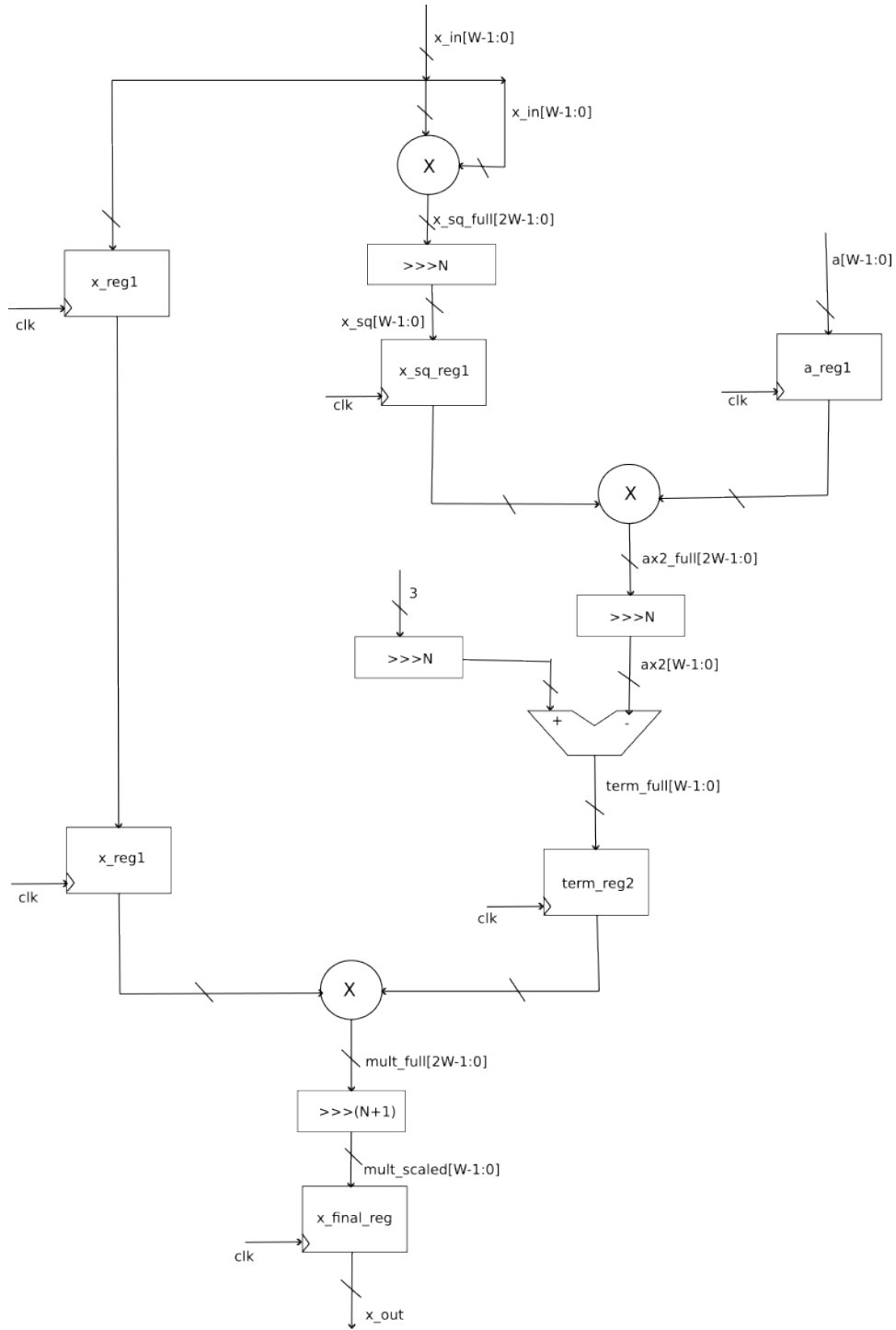


Fig. 4. nr\_stage\_pipeline

#### D. Pipelined Iteration Chain

The **top\_pipelined** module connects multiple NR stages:

- **x\_bus[]** – Carries intermediate  $x_i$  values
- **a\_bus[]** – Propagates input  $a$  through pipeline delays

Each stage introduces latency, and delay registers ensure proper synchronization of  $a$  with corresponding  $x_i$ .

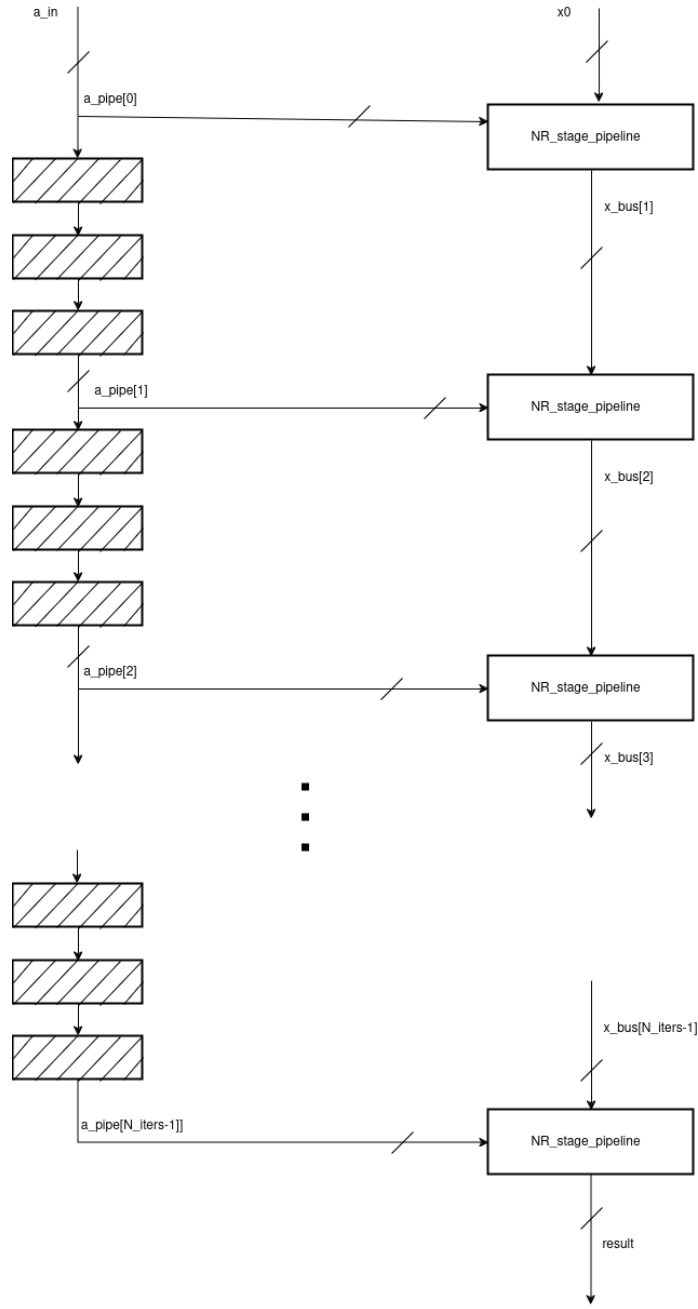


Fig. 5. NR Pipeline chain

### E. Top-Level Integration

The **top\_InvSqrt** module:

- Generates  $x_0$  using **initial\_approx**
- Aligns input  $a$  using delay registers
- Feeds both into the pipelined NR stages

After  $N\_ITERS$  iterations, the final result approximates:

$$x \approx \frac{1}{\sqrt{a}} \quad (3)$$

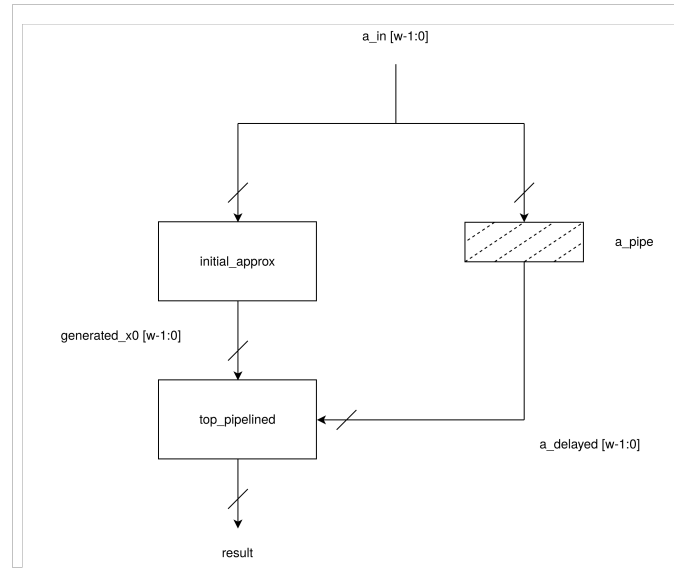


Fig. 6. top\_module

The design is fully pipelined, enabling high-throughput operation with one result produced per clock cycle after pipeline fill. The pipeline fills in  $(3 * NUM\_ITERS) + 1$  as each iteration is 3 staged and 1 delay extra is due to the delay in *initial\_approx* module.

### III. VERIFICATION AND OBSERVATION FROM TESTBENCH

The above design was tested using testbench, and a golden reference model was made using python with the same architectural design, to test the outputs of the design. Testbench takes an input file with 200 input values, which were generated on random using a separate python script. Then the outputs are matched with the python golden model's output for the same input values.

```

1 parameter N = 16;
2 parameter W = 32;
3 parameter N_ITERS = 4;
4 // Total Latency: 1 (Approx) + (3 * N_ITERS) (Pipeline)
5 localparam TOTAL_LATENCY = 1 + (N_ITERS * 3);
6 localparam inputs_count = 200;
7 reg clk, rst;
8 reg [W-1:0] a_in;
9 wire [W-1:0] result;
10 reg [W-1:0] a_history [0:TOTAL_LATENCY + 1];
11
12 reg [W-1:0] golden_mem [0:inputs_count - 1];
13 reg [W-1:0] input_mem [0:inputs_count-1];
14 integer out_ptr = 0;
15 integer inp_file, golden_out_file, r, i;
16
17 initial begin
18     inp_file = $fopen("NR_inputs.txt", "r");
19     for (i = 0; i < inputs_count; i = i + 1) begin
20         r = $fscanf(inp_file, "%d\n", input_mem[i]);
21     end
22     $fclose(inp_file);
23
24 end
25
26 initial begin
27     // golden_out_file = $fopen("NR_values.txt", "r");

```

```

28     golden_out_file = $fopen("NR_golden_values.txt", "r");
29     for (i = 0; i < inputs_count; i = i + 1) begin
30         r = $fscanf(golden_out_file, "%d\n", golden_mem[i]);
31     end
32     $fclose(golden_out_file);
33 end
34
35 initial clk = 0;
36 always #5 clk = ~clk;
37
38 // Instantiate the full integrated module
39 top_InvSqrt #(N, W, N_ITERS) dut (
40     .clk(clk), .rst(rst), .a_in(a_in), .result(result)
41 );
42
43 integer j, k;
44 always @(posedge clk) begin
45     a_history[0] <= a_in;
46     for (j = 1; j <= TOTAL_LATENCY + 1; j = j + 1) begin
47         a_history[j] <= a_history[j-1];
48     end
49 end
50
51 initial begin
52     // --- Initialization ---
53     rst = 1; a_in = 0;
54     repeat(10) @(posedge clk);
55     rst = 0;
56     @(posedge clk);
57
58     // --- Test Stream ---
59     for (k = 0; k < inputs_count; k = k + 1) begin
60         @(posedge clk);
61         a_in <= input_mem[k];
62     end
63     // --- End of Stream ---
64
65     @(posedge clk); a_in <= 0;
66     repeat (20) @(posedge clk);
67     $display("Testbench complete.");
68     $finish;
69 end
70
71 integer count_0;
72 integer diff;
73
74 initial begin
75     out_ptr = 0;
76     wait (rst == 0);
77     count_0 = 0;
78     repeat (TOTAL_LATENCY + 2) @(posedge clk);
79
80     forever begin
81         @(posedge clk);
82         if (out_ptr < inputs_count) begin
83             diff = (result > golden_mem[out_ptr]) ?
84                 (result - golden_mem[out_ptr]) :
85                 (golden_mem[out_ptr] - result);
86             if (diff == 0)
87                 count_0 = count_0 + 1;
88
89             $display("Time: %t | Input A: %d | Result: %d | Golden: %d | Diff: %d | Exact Match
: %d out of %d",

```

```

90         $time,
91         a_history[TOTAL_LATENCY-1],
92         result,
93         golden_mem[out_ptr],
94         diff,
95         count_0,
96         inputs_count
97     );
98     out_ptr = out_ptr + 1;
99 end
100 end
101 end

```

Listing 1. Testbench

|                     |         |          |       |         |        |         |        |       |   |              |            |     |
|---------------------|---------|----------|-------|---------|--------|---------|--------|-------|---|--------------|------------|-----|
| Time:               | 2125000 | Input A: | 57754 | Result: | 69812  | Golden: | 69812  | Diff: | 0 | Exact Match: | 188 out of | 200 |
| Time:               | 2135000 | Input A: | 32826 | Result: | 92601  | Golden: | 92601  | Diff: | 0 | Exact Match: | 189 out of | 200 |
| Time:               | 2145000 | Input A: | 55404 | Result: | 71277  | Golden: | 71277  | Diff: | 0 | Exact Match: | 190 out of | 200 |
| Time:               | 2155000 | Input A: | 25739 | Result: | 104574 | Golden: | 104574 | Diff: | 0 | Exact Match: | 191 out of | 200 |
| Time:               | 2165000 | Input A: | 24073 | Result: | 108133 | Golden: | 108133 | Diff: | 0 | Exact Match: | 192 out of | 200 |
| Time:               | 2175000 | Input A: | 6735  | Result: | 204434 | Golden: | 204434 | Diff: | 0 | Exact Match: | 193 out of | 200 |
| Time:               | 2185000 | Input A: | 40969 | Result: | 82888  | Golden: | 82888  | Diff: | 0 | Exact Match: | 194 out of | 200 |
| Time:               | 2195000 | Input A: | 839   | Result: | 579217 | Golden: | 579217 | Diff: | 0 | Exact Match: | 195 out of | 200 |
| Time:               | 2205000 | Input A: | 14109 | Result: | 141245 | Golden: | 141245 | Diff: | 0 | Exact Match: | 196 out of | 200 |
| Time:               | 2215000 | Input A: | 10226 | Result: | 165909 | Golden: | 165909 | Diff: | 0 | Exact Match: | 197 out of | 200 |
| Time:               | 2225000 | Input A: | 54700 | Result: | 71734  | Golden: | 71734  | Diff: | 0 | Exact Match: | 198 out of | 200 |
| Time:               | 2235000 | Input A: | 563   | Result: | 707077 | Golden: | 707077 | Diff: | 0 | Exact Match: | 199 out of | 200 |
| Time:               | 2245000 | Input A: | 23447 | Result: | 109567 | Golden: | 109567 | Diff: | 0 | Exact Match: | 200 out of | 200 |
| Testbench complete. |         |          |       |         |        |         |        |       |   |              |            |     |

Fig. 7. Comparison with Golden model

Testing of the design was also done by comparing the verilog architecture's output with the output of the python's *math* library's *sqrt()* function.

```

1 import math
2 N = 16
3 W = 2 * N
4 with open("NR_inputs.txt", "r") as f:
5     inputs = [int(line.strip()) for line in f if line.strip()]
6 def to_qN_N(x):
7     return int(round(x * math.pow(2, N)))
8 f = open("NR_golden_values.txt", "w")
9 for a in inputs:
10     val = a / float(math.pow(2, N))
11     inv_sqrt = 1 / math.sqrt(val)
12     q = to_qN_N(inv_sqrt)
13     # print(q)
14     f.write(f"{q}\n")
15 f.close()

```

Listing 2. Python Math.sqrt function test

|                         |         |          |       |         |        |         |        |       |   |              |            |     |
|-------------------------|---------|----------|-------|---------|--------|---------|--------|-------|---|--------------|------------|-----|
| Time:                   | 2125000 | Input A: | 30827 | Result: | 87423  | Golden: | 87423  | Diff: | 0 | Exact Match: | 99 out of  | 200 |
| Time:                   | 2135000 | Input A: | 14304 | Result: | 140279 | Golden: | 140279 | Diff: | 0 | Exact Match: | 100 out of | 200 |
| Time:                   | 2145000 | Input A: | 27105 | Result: | 101905 | Golden: | 101905 | Diff: | 0 | Exact Match: | 101 out of | 200 |
| Time:                   | 2155000 | Input A: | 19937 | Result: | 118821 | Golden: | 118820 | Diff: | 1 | Exact Match: | 101 out of | 200 |
| Time:                   | 2165000 | Input A: | 18403 | Result: | 123674 | Golden: | 123673 | Diff: | 1 | Exact Match: | 101 out of | 200 |
| Time:                   | 2175000 | Input A: | 5091  | Result: | 235136 | Golden: | 235136 | Diff: | 0 | Exact Match: | 102 out of | 200 |
| Time:                   | 2185000 | Input A: | 14825 | Result: | 137792 | Golden: | 137792 | Diff: | 0 | Exact Match: | 103 out of | 200 |
| Time:                   | 2195000 | Input A: | 8682  | Result: | 180058 | Golden: | 180057 | Diff: | 1 | Exact Match: | 103 out of | 200 |
| Time:                   | 2205000 | Input A: | 24044 | Result: | 108198 | Golden: | 108197 | Diff: | 1 | Exact Match: | 103 out of | 200 |
| Time:                   | 2215000 | Input A: | 62959 | Result: | 66864  | Golden: | 66864  | Diff: | 0 | Exact Match: | 104 out of | 200 |
| Time:                   | 2225000 | Input A: | 46577 | Result: | 77739  | Golden: | 77738  | Diff: | 1 | Exact Match: | 104 out of | 200 |
| Time:                   | 2235000 | Input A: | 53237 | Result: | 72713  | Golden: | 72713  | Diff: | 0 | Exact Match: | 105 out of | 200 |
| Time:                   | 2245000 | Input A: | 34010 | Result: | 89923  | Golden: | 89922  | Diff: | 1 | Exact Match: | 105 out of | 200 |
| Testbench complete.     |         |          |       |         |        |         |        |       |   |              |            |     |
| Max Diff = 32           |         |          |       |         |        |         |        |       |   |              |            |     |
| Final Exact Match = 105 |         |          |       |         |        |         |        |       |   |              |            |     |

Fig. 8. Comparison with python Math.sqrt function

As observed from Figure 8, the error margin is very small even when compared with the expected value of the square root.

For  $a = 2$ ,

$$\text{Error}\% = \frac{56}{5931642} \times 100 \approx 0.00094\%$$

and for  $a = 4$ ,

$$\text{Error}\% = \frac{32}{8388608} \times 100 \approx 0.00038\%$$

The testing was first don't for Q(8.8) format, which was then scaled up to Q(16.16) format for 32bit inputs. The error reduced slightly when the precision was increased.

This error keeps decreasing when the value of  $a$  increases, so the maximum observable error percentage for Q16.16 format was found to be  $\approx 0.001\%$ , which is approximately the same as  $2^{-16} \approx 0.0015\%$ . For larger inputs, the difference was at most  $\approx 0.0003\%$ . So, the observed error is typically within 1 LSB of the fixed-point representation, which is a totally negligible value in cases of DSP systems and many graphics approximations.

So, this tradeoff of small error margin for a simpler hardware and low latency, is a good tradeoff for modern systems.

#### IV. SIMULATION

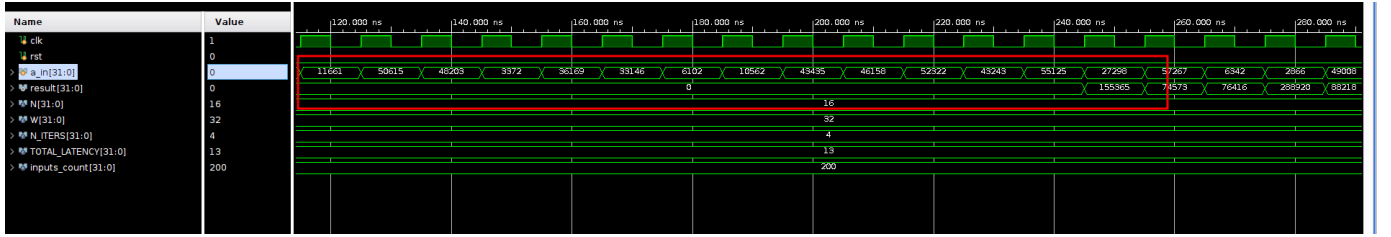


Fig. 9. Waveform of Pipelined design simulation

The waveform in Figure 9, the inputs enters the pipeline at  $T = 120ns$  and the output for it comes at  $T = 250ns$ . So, basically the input takes 13 cycles to generate into output. This 13 cycle latency comes from 3 cycle latency per iteration and 4 iterations in total plus 1 cycle latency in initial\_approx module.

$$\text{Latency} = (3 * \text{number\_of\_iterations}) + 1 = 12 + 1 = 13 \text{cycles}$$

The input is  $a = 11661$  in Q(16.16) format which is  $a = \frac{11661}{2^{16}} \approx 0.1779$  and the inverse square-root of the input is:  $\frac{1}{\sqrt{a}} = \frac{1}{\sqrt{0.1779}} \approx 2.2370 = 155365$  in Q(16.16) format. This matches the output obtained after the 13 cycles.

#### V. CONCLUSION

This project implemented a fixed-point inverse square root computation using the Newton–Raphson (NR) method in hardware. The design used a Q-format representation (initially Q8.8, later extended to Q16.16) and a pipelined architecture with three stages per iteration. An initial approximation was generated using a leading-one detector (LOD), followed by iterative refinement through NR stages. Key findings include:

- The pipeline latency matched the expected architecture
- Increasing precision from Q8.8 to Q16.16 significantly reduced error

The project tested the correctness of Initial Approximation Sensitivity, Leading One detection, and the final top module. It was also observed that the Newton-Raphson was especially suited for hardware implementation, due to faster convergence and simpler design.

#### REFERENCES

- [1] E. Libessart, M. Arzel, C. Lahuec, and F. Andriulli, "Scaling-less Newton-Raphson pipelined implementation for a fixed-point inverse square root operator," *2017 15th IEEE International New Circuits and Systems Conference (NEWCAS)*, Strasbourg, France, 2017, pp. 157–160, doi: 10.1109/NEWCAS.2017.8010129.
- [2] C. Lomont, "Fast inverse square root," 2003. [Online]. Available: <http://www.lomont.org/papers/2003/InvSqrt.pdf>